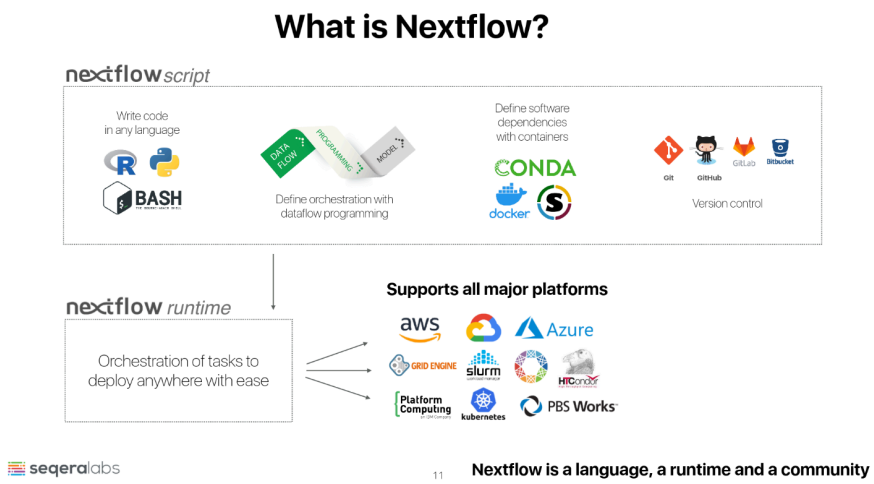
Reproducible, scalable bioinformatics workflows with nextflow and nf-core: Part 1

Author: Yucheng Zhang, Senior Bioinformatic Engineer, TTS Research Technology, yucheng.zhang@tufts.edu

Date: 2026-02-27

What is Nextflow?

Nextflow is a workflow system for creating scalable, portable, and reproducible workflows. It uses a dataflow programming model that simplifies writing parallel and distributed pipelines by allowing you to focus on data flow and computation. Nextflow can deploy workflows on a variety of execution platforms, including your local machine, HPC schedulers, and cloud. Additionally, Nextflow supports a range of compute environments, software container runtimes, and package managers, allowing workflows to be executed in reproducible and isolated environments.



Why nextflow?

Inefficient for loops

In [ ]:
```
cd /cluster/tufts/workshop/public/2026spring/nextflow/fastqs
```

In [ ]:
```
module load fastqc
# Run FastQC sequentially
mkdir -p fastqc_loop_results
```

```
time (
  for f in *.fastq.gz; do
      fastqc "$f" -o fastqc_loop_results
  done
)
```

In [ ]:
```
echo "params.reads = \"*.fastq.gz\"
params.outdir = \"fastqc_nf_results\"

process FASTQC {
    tag \"\$read\"
    publishDir params.outdir, mode: 'copy'

    beforeScript 'module load fastqc'

    input:
    path read

    output:
    path \"*_fastqc.{html,zip}\"

    script:
    \"\"\"
    fastqc \$read
    \"\"\"
}

workflow {
    read_ch = Channel.fromPath(params.reads)
    FASTQC(read_ch)
}" > fastqc.nf
```

In [ ]:
```
module load nextflow
time nextflow run fastqc.nf
```

How ro running nextflow pipelines?

Nextflow is designed to handle pipeline code seamlessly, whether it's sitting on your disk or hosted on GitHub.

Local Pipelines

To run a script you've written or a pipeline you've cloned manually to your current directory:

Command: `nextflow run main.nf`.

Best Practice: Ensure your main script is named main.nf (the Nextflow default) or define it in a manifest block within a nextflow.config file.

Remote Pipelines (GitHub)

Nextflow can pull and run pipelines directly from a repository without you needing to manually download them.

Simple Run: `nextflow run <organization>/<repository> (e.g., nextflow run nextflow-io/hello)`.

Specific Revisions: Use the **-r** flag to pin a specific version, branch, or commit for reproducibility: nextflow run `nf-core/rnaseq -r 3.22.0`.

In [ ]:
```
nextflow run nextflow-io/hello
```

You may notice a message like **Pulling nextflow-io/hello** in the log. This means that when you run a pipeline hosted on GitHub, Nextflow first downloads a local copy of that pipeline. On our cluster, the downloaded files are cached in your home directory under `$HOME/.nextflow`

In [ ]:
```
tree $HOME/.nextflow
```

In [ ]:
```
more $HOME/.nextflow/assets/nextflow-io/hello/main.nf
```

Profiles

In Nextflow, a profile is a set of configuration attributes that can be enabled with a single command-line flag, allowing you to switch between different execution environments easily. Profiles are essential for moving a pipeline from a local machine to a high-performance computing (HPC) cluster or a cloud environment without changing the underlying code.

Types of Profiles

Nextflow workflows typically utilize three categories of profiles to manage resource allocation and software dependencies:

- Core Profiles: These define the software management system, such as singularity, docker, or conda.
- Institutional Profiles: These are pre-configured sets of rules for specific clusters (e.g., `-profile tufts`) that include the correct scheduler (Slurm), partition names, and resource limits.

- Test Profiles: These allow users to run a pipeline with a minimal dataset to verify that the installation and configuration are working correctly.

The Tufts Institutional Profile

For users on the Tufts HPC, the tufts profile is specifically designed to handle the cluster's architecture:

- Job Submission: It automatically configures the Slurm workload manager, submitting individual pipeline tasks as child jobs. The **batch** partition is used as default for job submission.
- Resource Limits: It sets default maximums for memory (e.g., **120GB**), CPUs (e.g., **72**), and time (e.g., **168 hours**) to ensure jobs comply with cluster policies.
- Container Integration: It enables `Singularity` by default to handle dependencies.
- Dataset Access: It defines paths to shared resources, such as the igenomes_base directory, so users don't have to download their own igenomes reference genomes.

cache

Image Download Cache (`NXF_SINGULARITY_CACHEDIR`)

Nextflow downloads large Singularity/Apptainer images for every tool in a pipeline. By default, these are stored in a hidden folder in your home directory.

- Content: It stores `.img` or `.sif` files, which are read-only compressed files containing the entire operating system and software stack (like Samtools or R) needed for a task.
- Purpose: It prevents Nextflow from re-downloading massive container images from the internet every time you run a pipeline. Once an image is in this cache, all future runs will simply "point" to it.

You must redirect this by setting an environment variable in your `~/.bashrc` file.

In [ ]:
```
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/yourlab/user/singularity_cache
```

Work Directory Cache (`/work`)

The work directory is the "scratch space" where Nextflow orchestrates the actual execution of your pipeline.

- Content: It stores every intermediate file generated by every task in your pipeline (e.g., trimmed fastq files, aligned BAM files, or temporary indices).
- Purpose: It enables the **-resume** feature. Nextflow checks the "hashes" (unique fingerprints) of tasks in this directory to see if they have already been completed, allowing it to skip them in subsequent runs.
- Storage Impact: This directory grows extremely fast and can easily reach hundreds of gigabytes because it holds data for every step of every sample.
- Location: By default, it is created in your current launch directory as a folder named **work**.

Cache and Resume

resume

If you run a pipeline again with the `-resume` flag, Nextflow checks if that specific hash already exists in the `work` directory. If it finds a match, it "links" the previous results instead of running the code again.

clean cache

After a pipeline is completed with success, it's better to clean up work directory to save space.

You can remove the work directory completely by:

In [ ]:
```
rm -rf work
```

Another smarter method is to use `nextflow log` together with `nextflow clean`.

```
nf-training -> nextflow log
TIMESTAMP               DURATION        RUN NAME        STATUS  REVISION ID     SESSION ID                              COMMAND
2024-03-07 21:03:07     2.9s            clever_darwin   OK      3c3d5e1897      c6f83839-fb98-45af-9090-6807b02a1800    nextflow run hello.nf
2024-03-07 21:03:33     1.8s            chaotic_faggin  OK      86d466d737      9a963a51-3351-4c1a-8d7d-ed7643c11c44    nextflow run script1.nf
nf-training -> nextflow clean clever_darwin -f
Removed /workspace/gitpod/nf-training/work/f2/14d3a75f9b4c683bcf5e361931bcc9
Removed /workspace/gitpod/nf-training/work/ea/0cf312c156b549204e8b8b438739ed
Removed /workspace/gitpod/nf-training/work/f8/91d79e889abde3cf52c41e1a078320
nf-training ->
```

nf-core

nf-core is a community-driven project that provides a curated collection of high-

quality, peer-reviewed analysis pipelines built using the Nextflow workflow manager.

While Nextflow is the engine that handles the execution and parallelization of tasks, nf-core provides the standardized "blueprints" for specific types of biological data analysis (such as RNA-seq, ChIP-seq, or variant calling).

As of early 2026, there are over 140 nf-core pipelines at various stages of development. Some of the most widely used include:

- nf-core/rnaseq: For RNA sequencing analysis.
- nf-core/scrnaseq: For scRNA sequencing analysis.
- nf-core/atacseq: For ATAC-seq data processing.
- nf-core/ampliseq: For microbiome 16S rRNA analysis.

At Tufts, quite a few labs have integrated nf-core pipelines into their research, leveraging these standardized workflows to streamline data analysis, improve reproducibility, and facilitate collaboration across projects.

How to run it on nf-core pipelines on Tufts HPC?

Please do not run nf-core pipelines in $HOME. The large intermediate files generated will likely cause you to exceed your quota. Instead, utilize your research group’s project folder for all active runs.

In [ ]:
```
mkdir -p /cluster/tufts/rt/yzhang85/nf-core-demo ## change this your own folde
cd /cluster/tufts/rt/yzhang85/nf-core-demo
```

Load the module

At Tufts HPC, Singularity is recommended for handling software dependencies. Therefore, users need to load both the nextflow and singularity modules before running workflows.

In [ ]:
```
module load nextflow
module load singularity
module list
```

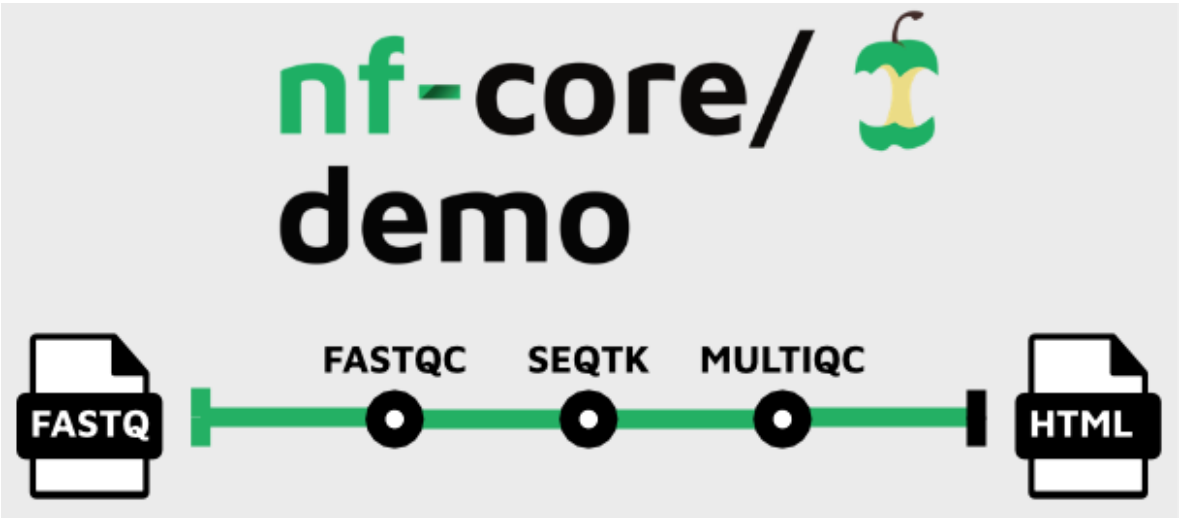
Define NXF_SINGULARITY_CACHEDIR

If `NXF_SINGULARITY_CACHEDIR` is already defined in your `.bashrc` (highly

recommended), you can safely skip the next step.

In [ ]:
```
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/rt/yzhang85/singularity_cache #
```

In the following sections, we will use demonstrate how to run nf-core pipelines on Tufts HPC with nf-core/demo, which is a simple nf-core style bioinformatics pipeline for workshops and demonstrations. It was created using the nf-core template and is designed to run quickly using small test data files.



.

nf-core pipelines provide a built-in **test** profile designed for quick validation of your setup. This profile runs the pipeline on a small test dataset with minimal computational requirements, allowing you to verify that Nextflow, containers (e.g. Singularity), and the execution environment are working correctly.

The test profile is especially useful when:

- Running a pipeline for the first time
- Testing a new computing environment or configuration
- Verifying container access and caching
- Debugging configuration or permission issues

singularity profile

When using this mode, all nextflow tasks are executed on a single compute node. No interaction with the scheduler, making it ideal for beginners. But all tasks share the resources of a single node, which can become a bottleneck.

In [ ]:
```
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/rt/yzhang85/singularity_cache
```

In [ ]:
```
nextflow run nf-core/demo -r 1.1.0 \
   -profile singularity,test \
   --outdir demo_out
```

Use the `-resume` flag when re-running a pipeline. This ensures that only failed or modified tasks are executed, while all successful steps are skipped and pulled from the cache.

In [ ]:
```
nextflow run nf-core/demo -r 1.1.0\
   -profile singularity,test \
   --outdir demo_out_singularity -resume
```

tufts profile

The tufts profile enables Nextflow to distribute tasks across the entire cluster, significantly accelerating your workflow through massive parallelism. However, please note that during periods of high cluster utilization, you may experience longer queue wait times before your jobs begin execution. Before running it, let's clean the previous run.

In [ ]:
```
nextflow run nf-core/demo -r 1.1.0\
   -profile tufts,test \
   --outdir demo_out_tufts
```

In [ ]:
```
nextflow log
```

In [ ]:
```
nextflow clean xxx -f
```

In [ ]:
```
sacct -u yzhang85
```

Example job scripts

Local mode

All nf-core tasks run on a single node. For this mode, you need to `-profile singularity`.

In [ ]:
```
#!/bin/bash

#SBATCH --time=00-48:00:00
#SBATCH -p batch
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c XX
#SBATCH --mem=XXG
#SBATCH --job-name nf-core
```

```
#SBATCH --output=%x-%J-%u.out
#SBATCH --error=%x-%J-%u.err
#SBATCH --mail-type=ALL
#SBATCH --mail-user=XXX@tufts.edu

module load nextflow
# Note: Redirecting cache to shared storage prevents $HOME quota issues
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/xxLab/$USER/nf-core/singularity

nextflow run nf-core/rnaseq -r 3.22.2 \
    --input samplesheet.csv --outdir output \
    --fasta ref.fasta --gtf ref.gtf --aligner star_salmon \
    -profile singularity \
    --max_memory XXGB --max_cpus XX
```

- **--max_cpus XX**: set the maximum number of CPU cores that any single process in the pipeline can request.
- **--max_memory XXGB** set the maximum RAM any single task can request.

tufts mode

nf-core pipelines can run tasks across multiple nodes in the cluster. To enable this distributed mode, use `-profile tufts`.

> **ℹ Tufts mode requires minimal resources**
> This parent nf-core pipeline script only orchestrates the submission of child Slurm jobs, so allocating **1 CPU cores** is sufficient.

In [ ]:

```
#!/bin/bash

#SBATCH --time=00-48:00:00
#SBATCH -p batch
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 1
#SBATCH --job-name nf-core
#SBATCH --output=%x-%J-%u.out
#SBATCH --error=%x-%J-%u.err
#SBATCH --mail-type=ALL
#SBATCH --mail-user=XXX@tufts.edu

module load nextflow
# Note: Redirecting cache to shared storage prevents $HOME quota issues
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/xxLab/$USER/nf-core/singularity

nextflow run nf-core/rnaseq -r 3.22.2 \
```

```
    --input samplesheet.csv --outdir output \
    --fasta ref.fasta --gtf ref.gtf --aligner star_salmon \
    -profile tufts
```

How to use other partitions

When using `-profile tufts`, the default partition is the public `batch` partition. If you would like your nf-core jobs to run on a different partition (for example, your lab's contributed partition), you can specify it with `--partition <PartitionName>`. Below is an example configuration to run jobs on the preempt partition.

Please note that you can still use `#SBATCH -p batch` for the partent job.

In [ ]:
```
#!/bin/bash

#SBATCH --time=00-48:00:00
#SBATCH -p batch
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 1
#SBATCH --job-name nf-core
#SBATCH --output=%x-%J-%u.out
#SBATCH --error=%x-%J-%u.err
#SBATCH --mail-type=ALL
#SBATCH --mail-user=XXX@tufts.edu

module load nextflow
# Note: Redirecting cache to shared storage prevents $HOME quota issues
export NXF_SINGULARITY_CACHEDIR=/cluster/tufts/xxLab/$USER/nf-core/singularity

nextflow run nf-core/rnaseq -r 3.22.2 \
    --input samplesheet.csv --outdir output \
    --fasta ref.fasta --gtf ref.gtf --aligner star_salmon \
    -profile tufts --partition preempt
```

> **ℹ Run jobs on other partitions**
> The parent (launcher) job can still run on the **batch** partition using **#SBATCH -p** batch. Only the nf-core child jobs will be submitted to the partition you specify with **--partition**.

To be continued

Reproducible, scalable bioinformatics workflows with nextflow and nf-core: Part 2